

Identificador de anomalías prenatales asistido por inteligencia artificial

Descripción general:

El presente trabajo propone un sistema de asistencia clínica basado en inteligencia artificial para la identificación de anomalías prenatales mediante imágenes de ultrasonido. El objetivo del modelo no es reemplazar al especialista médico, sino funcionar como una herramienta de apoyo orientada a disminuir errores humanos durante la interpretación de estudios prenatales.

Debido a la complejidad de definir explícitamente todas las posibles anomalías fetales, el enfoque adoptado consistió en entrenar un modelo capaz de aprender representaciones asociadas a ultrasonidos considerados normales, de manera que cualquier desviación significativa pudiera generar una alerta de posible anomalía.

Conjunto de datos

Se utilizó el dataset Fetal Planes DB, disponible públicamente en:

[Fetal Planes DB Dataset](#)

El conjunto contiene imágenes de ultrasonido fetal clasificadas en diferentes planos anatómicos.

Arquitectura del modelo

Se empleó un modelo DenseNet121 preentrenado sobre ImageNet como backbone convolucional. Durante esta etapa inicial de experimentación, el backbone fue completamente congelado y únicamente se reemplazó la última capa fully connected por una capa lineal de dimensión:

```
nn.Linear(1024, 6)
```

Las seis clases utilizadas fueron:

- Abdomen
- Brain
- Femur
- Thorax
- Other
- Anomaly

Configuración de entrenamiento:

El entrenamiento se realizó utilizando el optimizador Adam con una tasa de aprendizaje de (1×10^{-4}) . La función de pérdida utilizada fue CrossEntropyLoss. El tamaño de batch fue de 32 imágenes y el entrenamiento se ejecutó durante 10 épocas.

Con el objetivo de mejorar la capacidad de generalización del modelo, se aplicaron técnicas de data augmentation durante la fase de entrenamiento. Las transformaciones implementadas incluyeron rotaciones aleatorias, cambios de brillo, contraste y saturación, además de flips horizontales aleatorios.

```
transforms.RandomHorizontalFlip(p=0.5),
```

```
transforms.RandomRotation(10),
```

```
transforms.ColorJitter(
```

```
    brightness=0.1,
```

```
    contrast=0.1,
```

```
    saturation=0.2,
```

```
    hue=0.05
```

```
)
```

Problemas encontrados durante el desarrollo:

Durante la implementación se presentaron distintos problemas técnicos relacionados con PyTorch y el manejo del pipeline de entrenamiento.

Inicialmente se produjo un error asociado al uso incorrecto del parámetro transforms dentro de ImageFolder, el cual fue corregido utilizando transform.

Posteriormente se presentó un error relacionado con multiprocessing en Windows debido al uso de num_workers > 0 sin encapsular el código principal bajo if __name__ == "__main__". Como solución temporal se configuró:

```
num_workers = 0
```

También se detectó un error relacionado con incompatibilidad entre las etiquetas del dataset y el número de clases definido en la última capa lineal, provocando la excepción:

```
Assertion `t >= 0 && t < n_classes` failed
```

El problema se resolvió ajustando correctamente la dimensión de salida de la capa final.

Finalmente, durante la evaluación del modelo, se detectó un error asociado al uso incorrecto de:

```
torch.max(outputs, 6)
```

debido a que el tensor únicamente contenía dimensiones válidas 0 y 1. La corrección realizada fue:

```
torch.max(outputs, 1)
```

Resultados preliminares

La evolución de la función de pérdida durante el entrenamiento mostró una convergencia estable:

Época Loss

1	1.4403
2	1.0872
3	0.9140
4	0.8115
5	0.7368
6	0.6871
7	0.6467
8	0.6108
9	0.5864
10	0.5604

El modelo alcanzó una precisión final de 81.34% sobre el conjunto de prueba.

No se observaron explosiones numéricas ni aparición de valores NaN durante el entrenamiento, lo que sugiere una dinámica de optimización estable incluso con el backbone congelado.

Evaluación preliminar sobre anomalías

Como prueba exploratoria, se evaluó una imagen asociada a una deformación craneal compatible con strawberry skull. El modelo clasificó la imagen dentro de la categoría “Anomaly” con una confianza de 48.15%.



Aunque el sistema no fue entrenado específicamente para dicha patología, este comportamiento sugiere que el modelo logró capturar parcialmente características estructurales asociadas a patrones anatómicos anormales.

Limitaciones actuales

El sistema aún presenta múltiples limitaciones. No se implementaron mecanismos de interpretabilidad visual como Grad-CAM o mapas de atención, el backbone permanece completamente congelado y no se realizó evaluación clínica formal mediante métricas de sensibilidad o especificidad. Además, el tamaño del dataset continúa siendo relativamente reducido para aplicaciones médicas de alta confiabilidad.

Trabajo futuro

Como trabajo futuro se plantea realizar fine-tuning parcial del backbone, particularmente sobre denseblock4, implementar técnicas de interpretabilidad visual, analizar embeddings latentes y explorar enfoques de anomaly detection basados en representación en lugar de clasificación supervisada tradicional.

Implementación de interpretabilidad visual mediante Grad-CAM

Con el objetivo de aumentar la interpretabilidad del sistema y proporcionar evidencia visual sobre las regiones anatómicas utilizadas durante la toma de decisiones, se incorporó una técnica de visualización basada en mapas de activación (heatmaps) utilizando Grad-CAM (*Gradient-weighted Class Activation Mapping*).

El propósito de esta técnica consiste en resaltar espacialmente las regiones de la imagen sobre las cuales el modelo “considera” que existe una posible anomalía. Esto resulta

particularmente importante en aplicaciones biomédicas, donde la interpretabilidad del modelo puede ayudar al especialista clínico a comprender mejor el razonamiento detrás de una predicción.

Para implementar Grad-CAM fue necesario analizar la estructura interna completa de la red convolucional utilizada:

```
print(model.features)
```

Tras inspeccionar la arquitectura de DenseNet121, se identificó que el bloque convolucional más adecuado para extraer información semántica y espacial corresponde a `denseblock4`.

```
(denseblock4): _DenseBlock(
  (denselayer1): _DenseLayer(
    (norm1): BatchNorm2d(512)
    (relu1): ReLU(inplace=True)
    (conv1): Conv2d(512, 128, kernel_size=(1,1))
    (norm2): BatchNorm2d(128)
    (relu2): ReLU(inplace=True)
    (conv2): Conv2d(128, 32, kernel_size=(3,3), padding=(1,1))
  )
)
```

En esta sección de la red se generan nuevos mapas de características (*feature maps*) capaces de conservar simultáneamente:

- Información semántica de alto nivel.
- Localización espacial aproximada de estructuras anatómicas.

La selección de esta capa se fundamenta directamente en el trabajo de Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization, donde se establece que las últimas capas convolucionales representan el mejor compromiso entre compresión semántica y preservación espacial.

De acuerdo con el paper:

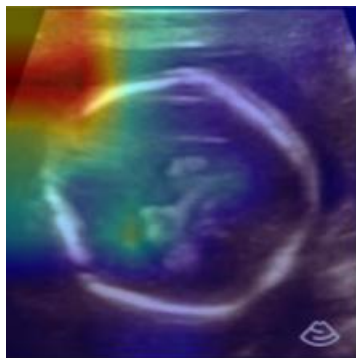
“We can expect the last convolutional layers to have the best compromise between high-level semantics and detailed spatial information.”

Grad-CAM utiliza la información de gradientes que fluye hacia la última capa convolucional para calcular la importancia relativa de cada mapa de activación respecto a una clase específica. Posteriormente, estos pesos son combinados para generar un heatmap que indica visualmente qué regiones contribuyeron más a la decisión final del modelo.

Conceptualmente, el procedimiento realizado fue:

1. Obtener los feature maps de denseblock4.
2. Calcular el gradiente de la clase predicha respecto a dichos feature maps.
3. Promediar espacialmente los gradientes para obtener pesos de importancia.
4. Combinar los feature maps ponderados.
5. Aplicar ReLU para conservar únicamente contribuciones positivas.
6. Superponer el heatmap resultante sobre la imagen original de ultrasonido.

La implementación de Grad-CAM permitió observar que el modelo concentra su atención sobre regiones anatómicas específicas durante la detección de anomalías, proporcionando así una primera aproximación hacia interpretabilidad visual en imágenes prenatales.



Aunque esta etapa aún es exploratoria y no constituye validación clínica, los resultados preliminares sugieren que el modelo no únicamente aprende patrones globales de clasificación, sino también representaciones espaciales asociadas a posibles alteraciones estructurales.

Adición de conjunto de validación y evaluación en test

Durante la implementación del modelo basado en **DenseNet121 preentrenada**, se realizó una mejora importante en la estructura experimental del pipeline, incorporando explícitamente un **conjunto de validación (validation set)** y un **conjunto de prueba (test set)**, con el objetivo de garantizar una evaluación más rigurosa y confiable del desempeño del modelo.

Inicialmente, el entrenamiento se realizaba únicamente con un conjunto de entrenamiento y una evaluación directa sobre el conjunto de test, lo cual no permitía monitorear el comportamiento del modelo durante el aprendizaje ni detectar posibles problemas de generalización.

Para corregir esto, el dataset de entrenamiento fue dividido internamente en dos subconjuntos:

- **Train set (80%)**: utilizado para la optimización de los parámetros del modelo mediante backpropagation.
- **Validation set (20%)**: utilizado durante el entrenamiento para evaluar el desempeño del modelo al final de cada época, permitiendo monitorear la evolución de la métrica de *validation accuracy* y detectar posibles problemas de overfitting.

Adicionalmente, se mantuvo un **test set independiente**, el cual no participa en el entrenamiento ni en la selección de hiperparámetros, y se utiliza exclusivamente para obtener una estimación final del rendimiento del modelo en datos completamente no vistos.

Esta separación permite una evaluación más robusta y acorde a prácticas estándar en aprendizaje profundo, especialmente en aplicaciones de visión por computadora en dominios médicos, donde la generalización del modelo es crítica.

resultado de la evaluación

Tras la incorporación de esta partición de datos, el modelo mostró el siguiente comportamiento:

- Incremento progresivo de la *validation accuracy* durante el entrenamiento.
- Convergencia estable del loss sin evidencia fuerte de overfitting.
- **Test accuracy final $\approx 83\%$** , consistente con el rendimiento observado en validación.

Conclusión de la mejora

La inclusión del validation set permitió:

- Monitorear el aprendizaje del modelo en tiempo real.
- Validar la estabilidad de la generalización.
- Obtener una evaluación final más confiable mediante el test set independiente.